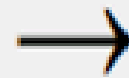


pandas.Series.value_counts

"Series.value_counts"
is used to return a
Series containing
counts of unique
values in pandas series.

```
0    apple
1    banana
2    apple
3    orange
4    banana
5    apple
dtype: object
```



```
apple    3
banana   2
orange   1
Name: count, dtype: int64
```

Contents

What Makes This Ebook Unique

Tips for Effective Use of This Ebook

Intoduction

[normalize](#)

[sort](#)

[ascending](#)

[dropna](#)

[normalize](#)

[bins](#)

[dropna](#)

[Author Biography](#)

[Other ebooks](#)

[Contacts and Social Media](#)

[Sources & References](#)

[Thank You!](#)

1st Edition

Why This E-book?

"The aim of this ebook is to give you the 'aha' moment right away at the start of learning a new concept."

- Practical step By Step Guide With Simple Examples
- Visual Illustrations and Interactive

- Simple Datasets
- Comprehensive Coverage (pandas Documentation used as reference)

Tips for Effective Use of This Ebook

Use Examples First: If you don't understand the text, go straight to the examples—they're self-explanatory. After reviewing them, return to the text to grasp the practical concepts.

Practice with Datasets: Copy the datasets provided and practice using them to reinforce your understanding.

Click on the blue links to go directly to the section you want to learn about.

Introduction

```
#import Libraries
import numpy as np
import pandas as pd
```

"Series.value_counts"
is used to return a
Series containing
counts of unique
values in pandas series.

```
0    apple
1    banana
2    apple
3    orange
4    banana
5    apple
dtype: object
```



```
apple    3
banana   2
orange    1
Name: count, dtype: int64
```

data.value_counts()

Code Snippet:

```
data = pd.Series(['apple', 'banana', 'apple', 'orange', 'banana', 'apple'])  
data
```

data:

```
0    apple  
1    banana  
2    apple  
3    orange  
4    banana  
5    apple  
dtype: object
```

"Series.value_counts"
is used to return a
Series containing
counts of unique
values in pandas series.

```
0    apple  
1    banana  
2    apple  
3    orange  
4    banana  
5    apple  
dtype: object
```

→

```
apple    3  
banana   2  
orange   1  
Name: count, dtype: int64
```

The "Series.value_counts" syntax

```
Series.value_counts(normalize=False, sort=True, ascending=False, bins=None,  
dropna=True)
```

The `normalize` parameter

When the "normalize" parameter is set to True, the object returned will contain the relative frequencies of the unique values.

The "normalize" parameter accepts Boolean, and defaults to False.

Without normalize(default):

data:

0 apple

1 banana

2 apple

3 orange

4 banana

5 apple

dtype: object

If False or
(Without normalize)
it returns the
raw counts of
each unique value in
the series.

apple 3
banana 2
orange 1
Name: count, dtype: int64

(normalize=True):

If True then the
object returned will
contain the relative
frequencies of the
unique values.

apple 0.500000
banana 0.333333
orange 0.166667
Name: proportion, dtype: float64

```
data.value_counts(normalize=False)  
or  
data.value_counts()
```

```
data.value_counts(normalize=True)
```

Code Snippet:

```
data = pd.Series(['apple', 'banana', 'apple',  
                 'orange', 'banana', 'apple'])  
data
```

→

data:

```
0    apple  
1    banana  
2    apple  
3    orange  
4    banana  
5    apple  
dtype: object
```

The **sort** parameter

The "sort" parameter accepts Boolean, and defaults to True.

It Sorts by frequencies when True

Preserve the order of the data when False.

data:

When sort=False:

(sort=True)(default):

0 orange

1 banana

2 apple



3 banana

4 apple

5 banana

dtype: object

If False:

It Preserves the
order of the
data when False.

```
orange    1
banana    3
apple     2
Name: count, dtype: int64
```

data.value_counts(sort=False)

If sort=True

It Sorts by
frequencies.

```
banana    3
apple     2
orange     1
Name: count, dtype: int64
```

data.value_counts(sort=True)

or

data.value_counts()

Without normalize(default):

data:

0 apple

1 banana

2 apple

3 orange

4 banana



If False or

(Without normalize)

it returns the
raw counts of
each unique value in
the series.

```
apple     3
banana     2
```

(normalize=True):

If True then the
object returned will
contain the relative
frequencies of the
unique values.

```
5    apple
dtype: object
```

```
orange    1
Name: count, dtype: int64

data.value_counts(normalize=False)
or
data.value_counts()
```

```
apple    0.500000
banana   0.333333
orange   0.166667
Name: proportion, dtype: float64

data.value_counts(normalize=True)
```

The ascending parameter

The "ascending" parameter accepts Boolean, and defaults to False.

It controls the order in which these counts are sorted.

data:

```
0    orange
1    banana
2    apple
3    banana
4    apple
5    banana
dtype: object
```

ascending=False(default):

```
banana    3
apple     2
orange     1
Name: count, dtype: int64

data.value_counts(ascending=False)
or
data.value_counts()
```

ascending=True:

```
orange    1
apple     2
banana    3
Name: count, dtype: int64

data.value_counts(ascending=True)
```

@Kichere The Data Scientist

Code Snippet:

```
data = pd.Series([ 'orange', 'banana', 'apple',  
                  'banana', 'apple', 'banana'])
```

data

data:

```
0    orange  
1    banana  
2     apple  
3    banana  
4     apple  
5    banana  
dtype: object
```



The **dropna** parameter

The "dropna" parameter accepts Boolean, and defaults to True.

If True , it doesn't count rows that have missing values."

data:

```
0    1.0  
1    2.0  
2    NaN  
3    2.0  
4    3.0
```

dropna=False:

```
2.0    2  
NaN     2  
3.0     2
```



ascending=True (default):

```
2.0    2  
3.0    2  
1.0    1
```

Name: count, dtype: int64

```
5    NaN
```

```
1.0    1
```

```
Name: count, dtype: int64
```

```
6    3.0
```

```
dtype: float64
```

```
data.value_counts(dropna=False)
```

```
data.value_counts(dropna=True)
```

```
or
```

```
data.value_counts()
```

Code Snippet:

```
data = pd.Series([1, 2, np.nan, 2, 3, np.nan, 3])
```

```
data
```

→

data:

```
0    1.0
```

```
1    2.0
```

```
2    NaN
```

```
3    2.0
```

```
4    3.0
```

```
5    NaN
```

```
6    3.0
```

```
dtype: float64
```

The `bins` parameter

The "bin" parameter allows for grouping numeric values into half-open bins rather than simply counting them.

It accepts integer value, and it is optional

*#Half-open bins refer to intervals where the lower bound is inclusive, and the upper bound is exclusive.
#This means that a value that falls exactly on the lower bound is included in the bin, but a value that falls
#exactly on the upper bound is excluded.*

#For example, consider the interval $[0, 5)$, which is half-open:

#Inclusive: The value 0 is included in the bin.

#Exclusive: The value 5 is not included in the bin.

data:

2	2
1	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1

Name: count, dtype: int64

Count Values Without Bins:

2	2
1	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1

Name: count, dtype: int64

data.value_counts()

Use bins to Group Data:

$(-0.001, 3.0]$ 4

$(6.0, 10.0]$ 4

$(3.0, 6.0]$ 3

Name: count, dtype: int64

data.value_counts(bins=[0, 3, 6, 10])

Sources & References

[pandas.Series.value_counts Documentation](https://pandas.pydata.org/docs/reference/api/pandas.Series.value_counts.html) (

https://pandas.pydata.org/docs/reference/api/pandas.Series.value_counts.html)

Other E-books on pandas

Download (https://github.com/kicherethedatascientist/My_e-Books)

Author Biography

Kichere Magubu is a data enthusiast and content creator.

Contacts and Social Media

Kichere Magubu

 **Dar es salaam, Tanzania**

 **Kichere Magubu**

 **Kichere The Data Scientist**

 **KichereTheDataScientist**

 **KichereTheDataScientist**

 **kicherethedatascientist@gmail.com**

 **kicherethedatascientist**

 **+255 654 729 851**

Thank You!

Thank you for reading this e-book! If you found it valuable, please consider leaving an honest review. Your feedback and support mean a lot to me!

```
#print("The cell to convert jupyter notebook to html")  
!jupyter nbconvert --to hide_code_html "Series.value_counts.ipynb"
```